Department of Computer Science and Engineering

Database Management Systems

Lab 08 | Isolation Level in Transaction Management

E-Commerce Order & Inventory Management – A Case Study

Objective

The objective of this experiment is to demonstrate how MySQL transaction isolation levels affect concurrent operations in an e-commerce order and inventory database.

Case Study

An online shopping platform maintains customers, products, and orders. Customers place orders for products, while inventory quantities are updated as purchases are processed. Multiple database sessions may access the same records at the same time. Transaction isolation is therefore important for preventing inconsistent reads and controlling concurrent updates.

The demonstrations use the following entities: Customer, Product, and Orders. The Product table contains inventory information, while Orders records purchases and their status.

Outcome

The experiment covers:

- Checking the current and global MySQL transaction isolation level.
- Creating and populating an e-commerce database.
- Demonstrating a dirty read using READ UNCOMMITTED.
- Demonstrating visibility of committed changes using READ COMMITTED.
- Demonstrating consistent repeated reads using REPEATABLE READ.
- Demonstrating blocking of concurrent inserts under SERIALIZABLE.

Before You Begin

- Run the two-session demonstrations in separate MySQL client windows or terminals. Session 1 runs Transaction A and Session 2 runs Transaction B; keep both windows open from Task 3 to Task 6.
- Before executing any command, set the prompt in the MySQL command line client so that your identity and the active database are visible on every line: prompt \u@\h

[\d]>_ (connect with a MySQL account named <SRN>_<NAME> so that \u prints it). Type each statement on one line so that no continuation line appears without the prompt.

- Keep the first transaction open whenever the instructions explicitly ask you to observe concurrency.
- Do not close or commit the first session before capturing the required screenshot.
- Use ROLLBACK or restore statements when a demonstration changes data that should remain unchanged for later experiments.
- If a lock wait occurs, allow the intended transaction to complete or use the MySQL lock-wait timeout behavior shown by your environment.

Deliverables

- A single PDF containing, in order, each of the 17 questions with the SQL you executed and a screenshot of its output, plus the SQL file with the complete code which you executed. Label each screenshot with its question number and session.
- File names:
- <SRN>_TransactionIsolation_Lab08.pdf
- <SRN>_TransactionIsolation_Lab08.sql

Tasks to be executed:

- **Task 1:** Check MySQL isolation level — Question 1.1
- **Task 2:** Database setup and sample data — Questions 2.1 – 2.3
- **Task 3:** READ UNCOMMITTED — Questions 3.1 – 3.2
- **Task 4:** READ COMMITTED — Questions 4.1 – 4.3
- **Task 5:** REPEATABLE READ — Questions 5.1 – 5.3
- **Task 6:** SERIALIZABLE — Questions 6.1 – 6.4
- **Task 7:** Verification queries — Question 7.1
- 17 questions in total; every question uses the ecommerce_db database created in Task 2.

Schema Reference

The questions below refer only to the tables and columns listed here. Use these names exactly.

This is the schema supplied with the case study.

Table	Columns	Primary key
Customer	customer_id, full_name, email, city	customer_id
Product	product_id, product_name, category, price, stock_quantity, status (index idx_category on category)	product_id
Orders	order_id, customer_id, product_id, quantity, total_amount, order_status	order_id

All primary keys are INT AUTO_INCREMENT. Orders.customer_id references Customer(customer_id) and Orders.product_id references Product(product_id). status is ENUM('Active','Inactive'); order_status is ENUM('Pending','Confirmed','Cancelled').

Database Setup

Copy and execute the following commands in MySQL.

```
CREATE DATABASE ecommerce_db;
USE ecommerce_db;

CREATE TABLE Customer (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    city VARCHAR(50) NOT NULL
);

CREATE TABLE Product (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT NOT NULL,
    status ENUM('Active','Inactive') DEFAULT 'Active',
    INDEX idx_category (category)
);

CREATE TABLE Orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
    order_status ENUM('Pending','Confirmed','Cancelled') DEFAULT 'Pending',
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
```

Insert Sample Data

```
INSERT INTO Customer (full_name, email, city)
VALUES
('Ananya Rao', 'ananya.rao@example.com', 'Bengaluru'),
('Karan Mehta', 'karan.mehta@example.com', 'Mumbai'),
('Sofia Khan', 'sofia.khan@example.com', 'Delhi');

INSERT INTO Product (product_name, category, price, stock_quantity, status)
VALUES
('Laptop Pro 14', 'Electronics', 85000.00, 20, 'Active'),
('Wireless Headphones', 'Electronics', 6500.00, 35, 'Active'),
('Office Chair', 'Furniture', 12000.00, 12, 'Active'),
('Mechanical Keyboard', 'Electronics', 4500.00, 25, 'Active');

INSERT INTO Orders (customer_id, product_id, quantity, total_amount, order_status)
VALUES
(1, 1, 1, 85000.00, 'Pending'),
(2, 2, 2, 13000.00, 'Confirmed'),
(3, 3, 1, 12000.00, 'Confirmed');
```

Verify the initial data:

```
SELECT * FROM Customer;
SELECT * FROM Product;
SELECT * FROM Orders;
```

Task 1: Check MySQL Isolation Level

1.1 Display the transaction isolation level that applies to your current session, and the global (server-wide) default isolation level that every new session starts with.

Task 2: Database Setup and Sample Data

2.1 Create the ecommerce_db database, make it the active database, and create the Customer, Product and Orders tables exactly as supplied with the case study.

2.2 Insert the supplied customers, products and orders into the three tables and record the number of rows inserted into each table.

2.3 Display the complete contents of the Customer, Product and Orders tables to verify the initial data.

Task 3: READ UNCOMMITTED

Transaction A changes the stock quantity of a product but does not commit. Transaction B reads the same product using READ UNCOMMITTED.

3.1 In Session 1 (Transaction A), set the isolation level to READ UNCOMMITTED, start a transaction and reduce the stock quantity of product 1 to 5. Do not issue COMMIT.

3.2 From a second session (Transaction B), set READ UNCOMMITTED, start a transaction and read the product_id, product_name and stock_quantity of product 1 while Transaction A is still uncommitted. Determine whether Transaction B can see the changed stock, then roll back both transactions.

Task 4: READ COMMITTED

Transaction A reads a product's stock. Transaction B changes the stock and commits. Transaction A reads the same product again.

4.1 In Session 1, set READ COMMITTED, start a transaction and read the stock_quantity of product 1.

4.2 In Session 2, set READ COMMITTED, start a transaction, change the stock_quantity of product 1 to 18 and commit.

4.3 In Session 1, without ending the transaction, read the stock_quantity of product 1 again and record the value. Then commit Transaction A.

Task 5: REPEATABLE READ

Transaction A reads the details of all active Electronics products. Transaction B changes one product's price and commits. Transaction A repeats the same read within its transaction.

5.1 In Session 1, set REPEATABLE READ, start a transaction and display the product_id, product_name, price and stock_quantity of all active Electronics products.

5.2 In Session 2, set REPEATABLE READ, start a transaction, change the price of product 2 to 7000.00 and commit.

5.3 In Session 1, repeat exactly the same query within the same transaction and record the price shown for product 2. Then commit Transaction A.

Task 6: SERIALIZABLE

Transaction A checks the current set of Active Electronics products. Transaction B attempts to insert another Active Electronics product.

6.1 In Session 1, set SERIALIZABLE, start a transaction and display the product_id, product_name, category and status of all active Electronics products. Keep Session 1 open.

6.2 In Session 2, set SERIALIZABLE, start a transaction and insert a new active Electronics product, USB-C Dock (price 8000.00, stock 15), while Session 1 remains open. Observe what happens to the INSERT.

6.3 Return to Session 1 and commit Transaction A.

6.4 Return to Session 2, record what happened to the INSERT after Session 1 committed, and commit Transaction B.

Task 7: Verification Queries

Write the queries that verify the final database state.

7.1 Display all products, and display each order with its customer name, product name, quantity, total amount and order status by joining Orders, Customer and Product.

Observation and Comparison

Complete the Observation column from your own executions.

Isolation Level	Main Demonstration	Observation	Concurrency
READ UNCOMMITTED	Dirty read		Highest concurrency
READ COMMITTED	Committed-value visibility		High
REPEATABLE READ	Consistent repeated read		Moderate
SERIALIZABLE	Blocking concurrent range operation		Lowest concurrency

The observations above are based on the e-commerce scenarios and the transaction behavior demonstrated by the commands in this lab. Actual timing and lock-wait output can vary with the MySQL version, storage engine, indexes, and execution plan.